

ASIGNATURA DOSW	TEMAS SOLID · Patrones de Diseño	MODALIDAD Individual	NOMBRE / FECHA Juan David Munar – 13/08/26
--------------------	-------------------------------------	-------------------------	---

Objetivo: El propósito de este taller es desarrollar la capacidad de análisis y diseño de soluciones de software mediante la identificación de violaciones SOLID, la selección de patrones adecuados y la justificación técnica de decisiones de arquitectura. La idea NO es memorizar definiciones, sino reconocer problemas reales de diseño y proponer soluciones fundamentadas.

Reglas: Para cada caso:

- (1) Analice la situación con detenimiento.
- (2) Identifique el problema principal.
- (3) Determine el principio SOLID involucrado.
- (4) Justifique técnicamente su respuesta.
- (5) Proponga una solución concreta con modificaciones de diseño cuando sea necesario.

REFERENCIA RÁPIDA — PRINCIPIOS SOLID

S Single Responsibility <i>Una clase = Una razón de cambio</i>	O Open / Closed <i>Abierto para extender, cerrado para modificar</i>	L Liskov Substitution <i>Subclases sustituibles sin romper el contrato</i>	I Interface Segregation <i>Interfaces pequeñas y específicas</i>	D Dependency Inversion <i>Depender de abstracciones, no de concreciones</i>
--	--	--	--	---



PARTE I — IDENTIFICANDO VIOLACIONES A SOLID

Detectar qué principio se rompe y por qué — 5 casos

#1

El Sistema de Facturación

Violacion de SOLID

Una empresa tiene una única clase `InvoiceManager` que lo hace todo: calcula impuestos, genera PDF, envía correos y persiste datos. Cada vez que cambia la legislación tributaria, el formato del PDF o el proveedor de correo, esta clase debe modificarse.

```
public class InvoiceManager {  
    public void calculateTaxes() { } lógica tributaria  
    public void generateInvoicePDF() { } formato de presentación  
    public void sendInvoiceByEmail() { } canal de comunicación  
    public void saveInvoiceDatabase() { } persistencia  
}
```

⚠ PRINCIPIO VIOLADO

Single responsibility

¿POR QUÉ?

La única clase es la que se encarga de todo, como dice la definición dada "Una razón de cambio". Al tener tantas funciones, de una forma u otra va a tener que cambiar si o si, haciendo que no sea sostenible y aumenten los errores colaterales.

#2**La Aplicación de Transporte****Violacion de SOLID**

Un sistema diseñado para transporte terrestre crece para incluir medios acuáticos. Al agregar Boat, el desarrollador lanza una `UnsupportedOperationException` en `move()`, rompiendo el contrato que el cliente esperaba de cualquier `Vehicle`.

```
class Vehicle {  
    public void move() { } // contrato: todo vehículo puede moverse  
}  
  
class Car    extends Vehicle { } // ✓ OK  
class Bicycle extends Vehicle { } // ✓ OK  
class Airplane extends Vehicle { } // ✓ OK (aunque no es terrestre...)  
  
class Boat extends Vehicle {  
    @Override  
    public void move() {  
        throw new UnsupportedOperationException(); // X ROMPE EL CONTRATO  
    }  
}
```

⚠ PRINCIPIO VIOLADO

Liskov substitution

¿POR QUÉ?

El bote hereda de vehículo, pero al llamar a su método lanza una excepción de operación no soportada. La definición dice "subclases sustituibles sin romper el contrato". Al lanzar ese error, el bote rompe el comportamiento esperado, causando fallos si el sistema intenta usarlo como un vehículo genérico.

#3

El Sistema de Notificaciones

Violacion de SOLID

NotificationService centraliza toda la lógica de envío con un gran bloque de if-else por tipo de canal. Agregar WhatsApp requirió abrir la clase y tocarla. Agregar Telegram o Push Notification requerirá lo mismo.

```
public class NotificationService {
    public void sendNotification(String type) {
        if (type.equals("EMAIL")) {
            // lógica email - proveedor puede cambiar
        }
        if (type.equals("SMS")) {
            // lógica SMS - puede cambiar operadora
        }
        if (type.equals("WHATSAPP")) {
            // lógica WhatsApp - API puede cambiar
        }
        // para agregar PUSH o TELEGRAM: abrir y modificar esta clase
    }
}
```

△ PRINCIPIO VIOLADO

Open / Closed

¿POR QUÉ?

El propio enunciado nos dice que, al agregar nuevas funcionalidades, por la forma en que está construido el sistema se va a tener que abrir la clase para ir modificándola según sea necesario, haciéndola para nada extensible.

#4

El Cajero Inteligente

Violacion de SOLID

SmartATM define 5 operaciones en una sola interfaz. Los cajeros antiguos solo retiran dinero, pero al implementar la interfaz se ven forzados a implementar (aunque sea vacío o lanzando excepción) métodos que no usan ni necesitan.

```
public interface SmartATM {
    void withdraw();        // cajero básico: ✓
    void deposit();         // cajero básico: X no soporta
    void printStatement();  // cajero básico: X no soporta
    void biometricValidation(); // cajero básico: X no tiene lector
    void cryptocurrencyTransfer(); // cajero básico: X nunca tendrá
}
```

// El cajero básico debe implementar métodos que jamás usará ↑

⚠ PRINCIPIO VIOLADO

Interface Segregation.

¿POR QUÉ?

Obligan al cajero básico a implementar métodos que nunca va a usar, como biometría o criptomonedas. La definición exige "interfaces pequeñas y específicas". Aquí tenemos una interfaz gigante que fuerza dependencias inútiles.

PREGUNTAS GUÍA

1. ¿Cuántas interfaces más cohesivas propondría en lugar de SmartATM? Nómbralas y asígneles sus métodos.

Propongo 5 interfaces específicas, guiándonos por la pista del ejercicio:

- Withdrawable -> método: withdraw()
- Depositable -> método: deposit()
- StatementPrintable -> método: printStatement()
- BiometricCapable -> método: biometricValidation()
- CryptoCapable -> método: cryptocurrencyTransfer()

2. ¿Qué problemas concretos genera que BasicATM tenga que implementar cryptocurrencyTransfer()?

Genera código basura por así decirlo. Además, si el día de mañana el método de criptomonedas cambia, nos obliga a modificar y recompilar BasicATM, arriesgando un código que ya funcionaba bien y que ni siquiera usa esa característica.

3. Si se agrega un cajero multdivisa, ¿qué interfaz(ces) debería implementar? ¿Tendría que tocar BasicATM?

Se crearía una nueva interfaz y cajero implementaría esa junto con las básicas que necesite. No tendría que tocar BasicATM para nada ya que cada cajero arma sus funciones de manera independiente.

4. ¿Cómo el ISP facilita agregar nuevas funcionalidades sin romper implementaciones existentes?

Porque en lugar de una interfaz vieja, creas una interfaz nueva y pequeña solo para la nueva funcionalidad. Solo las clases que necesiten esa función la implementarán, dejando completamente intactas a las demás clases que seguirán operando sin riesgo de romperse.



Pista: Piense en interfaces como: Withdrawable, Depositable, StatementPrintable, BiometricCapable, CryptoCapable. Un SmartATM moderno implementaría todas; BasicATM solo la primera.

#5

El Sistema de Pagos

Violacion de SOLID

PaymentProcessor instancia directamente PaypalGateway en su constructor. La clase de alto nivel (procesamiento de pagos) depende de una implementación concreta de bajo nivel (gateway específico). Cambiar a Stripe o Wompi requiere modificar PaymentProcessor.

```
public class PaymentProcessor {
    // Dependencia hardcodeada: acoplamiento total
    private PaypalGateway gateway = new PaypalGateway();

    public void processPayment(double amount) {
        gateway.pay(amount); // imposible cambiar sin tocar esta clase
    }
}

// Para soportar Stripe, MercadoPago o Wompi → modificar PaymentProcessor
```

⚠ PRINCIPIO VIOLADO

Dependency Inversion.

¿POR QUÉ?

El procesador está instanciando directamente a PaypalGateway con un new. La regla manda depender de abstracciones. Si queremos cambiar a Stripe o wompi, nos tocará reescribir la clase porque está totalmente acoplada a una implementación específica.

☑ PARTE II — ¿QUÉ PRINCIPIO DEBERÍA APLICARSE?

Identificar el principio correcto para cada situación — 5 casos

#6

E-Commerce con Múltiples Medios de Pago

Aplicación de SOLID

Una tienda online maneja carrito, inventario, cupones, métodos de pago y envíos. El equipo sabe que constantemente aparecen nuevos medios de pago (criptomonedas, PSE, Nequi, etc.) y teme que cada adición rompa el flujo de compra existente.

✓ PRINCIPIO A APLICAR

Open / Closed.

BENEFICIO CLAVE

Si el día de mañana agregamos Nequi o Criptomonedas es decir extendiendo, vamos a crear clases nuevas sin tocar el código central,

evitando dañar por accidente lo que ya esta estable.

#7

Plataforma de Streaming — Tipos de Usuario

Aplicación de SOLID

La plataforma tiene usuarios gratuitos, premium y familiares. Todos deben poder reproducir contenido con el mismo flujo. Sin embargo, los usuarios premium pueden descargar, los familiares pueden crear perfiles secundarios, y los gratuitos tienen límite de reproducción.

✓ PRINCIPIO A APLICAR

Liskov Substitution.

BENEFICIO CLAVE

Garantiza que la plataforma pueda tratar a todos los usuarios como un usuario base para el flujo de reproducción, sin que características exclusivas de cada usuario rompan la aplicación.

#8

Sistema de Sensores IoT

Aplicación de SOLID

Los sensores pueden enviar información mediante MQTT, HTTP o WebSocket. El sistema central de procesamiento debe recibir datos sin importar el protocolo de transporte. En el futuro podría aparecer un protocolo nuevo (CoAP, LoRaWAN, etc.).

✓ PRINCIPIO A APLICAR

Dependency Inversion.

BENEFICIO CLAVE

El sistema central no tiene por qué saber si el dato viene por MQTT o HTTP. Al depender de una abstracción genérica, podemos inyectar nuevos protocolos en el futuro sin modificar el motor de procesamiento.

#9

Aplicación Bancaria — Interfaz Monolítica

Aplicación de SOLID

BankUser tiene más de 30 métodos. Clientes usan 10, gerentes usan 15 y auditores usan 5. Cuando un método de BankUser cambia de firma para servir a los gerentes, todos los auditores y clientes deben recompilarse aunque el cambio no los afecte.

✓ PRINCIPIO A APLICAR

Interface Segregation.

BENEFICIO CLAVE

Rompe la interfaz gigante de 30 métodos en partes pequeñas. Así, si los gerentes necesitan un cambio en sus métodos, no obligas al cliente o al auditor a recompilar código que ni les importa ni usan. Es decir cada uno usa lo que necesita realmente.

#10

CourseManager — La Clase Que Lo Hace Todo

Aplicar: SRP

CourseManager crea cursos, envía correos, genera certificados, calcula estadísticas y administra profesores. Cada vez que cambia la plantilla de un certificado, el equipo de marketing debe coordinar con el equipo de estadísticas para no romperse entre sí.

✓ PRINCIPIO A APLICAR

Single Responsibility

BENEFICIO CLAVE

Evita que la clase lo haga todo. Separar correos, certificados y estadísticas en clases distintas asegura que un cambio visual que pida Marketing no vaya a tumbar el cálculo de notas de los estudiantes.



PARTE III — IDENTIFICANDO PATRONES DE DISEÑO

Seleccionar el patrón correcto y justificarlo — 8 casos

#11

El Único Administrador

Patrones de Diseño

La aplicación necesita gestionar la configuración global del sistema con un único objeto compartido por toda la aplicación. Si se crean múltiples instancias, podrían existir configuraciones inconsistentes entre módulos.

CATEGORÍA
Creacional

PATRÓN
Factory Method.

¿POR QUÉ?

La fábrica controlaría la lógica internamente para devolver siempre una instancia evitando que los módulos hagan un new directo y desincronicen el sistema.

#12

Múltiples Formas de Pago

Patrones de Diseño

Una tienda puede cobrar con tarjeta, PSE, PayPal o Nequi. El flujo de compra (seleccionar productos, confirmar, pagar) es siempre el mismo; solo cambia el algoritmo de pago según el método elegido.

CATEGORÍA
Comportamiento

PATRÓN
Strategy

¿POR QUÉ?

Tiene muchas clases similares que sólo se diferencian en la forma en que ejecutan el algoritmo de pago

#13

Generación de Reportes

Patrones de Diseño

El sistema genera reportes en PDF, Excel o CSV según la configuración del cliente. La lógica que solicita el reporte no debería conocer los detalles de construcción de cada formato.

CATEGORÍA
Creacional

PATRÓN
Abstract Factory

¿POR QUÉ?

Oculto la complejidad de construir cada tipo de reporte. El sistema solo pide el reporte y la fábrica se encarga de instanciar la clase concreta y entregarla lista para usar.

#14

Sistema de Eventos — Cambio de Estado de Pedido

Patrones de Diseño

Cuando un pedido cambia de estado, necesita notificar a inventario, correo, auditoría y notificación push. Agregar un nuevo suscriptor (como facturación automática) no debería requerir modificar el pedido.

CATEGORÍA

Comportamiento

PATRÓN

Command

¿POR QUÉ?

Cuando el pedido cambia de estado, simplemente ejecuta una lista de comandos, desacoplando el pedido de los servicios que reciben la alerta.

#15

Integración con el Banco Antiguo

Patrones de Diseño

El sistema moderno usa la interfaz `PaymentProcessor` (`modernpay`, `amount`). El banco antiguo expone `LegacyBankService` con métodos `executeTransaction(account, cents)` y `verifyBalance(account)`. No se puede modificar el sistema legado.

CATEGORÍA

Estructural

PATRÓN

Adapter

¿POR QUÉ?

Envuelve los métodos antiguos del banco para que el sistema moderno los pueda consumir usando la interfaz nueva (`modernpay`), sin tener que reescribir el sistema legado.

#16

Mensajes en una App de Chat

Patrones de Diseño

Los usuarios pueden enviar mensajes de texto, voz o video. Cada tipo puede tener distintos algoritmos de compresión (MP3, AAC para voz; H.264, HEVC para video). El tipo de mensaje y el algoritmo de compresión son dimensiones independientes que pueden combinarse.

CATEGORÍA

Estructural

PATRÓN

Bridge

¿POR QUÉ?

Separa el tipo de mensaje de su algoritmo de compresión. Si no se separan, se crea una combinación inmanejable de subclases por cada tipo y formato.

#17 Construcción de Vehículos Configurables

Creacional

El concesionario fabrica carros, camiones y motos. Cada uno tiene configuraciones opcionales: motor, transmisión, color, GPS, sonido, asistencia de carretera, etc. Un constructor con 15 parámetros es imposible de usar correctamente.

CATEGORÍA
Creacional

PATRÓN
Builder

¿POR QUÉ?

Permite construir objetos complejos paso a paso con configuraciones opcionales, evitando un constructor gigante y confuso con 15 parámetros.

#18 Sistema de Seguridad — Validaciones en Cadena

Patrones de Diseño

Un usuario intenta acceder a un recurso. La solicitud pasa por: autenticación, validación de rol, validación de permisos y validación geográfica. En algunos entornos hay validaciones adicionales (MFA, dispositivo confiable). Cada validación decide si pasa al siguiente o rechaza.

CATEGORÍA
Comportamiento

PATRÓN
Chain of Responsibility

¿POR QUÉ?

El programa debe procesar distintos tipos de solicitudes de varias maneras, cada uno de los manejadores procesa si se queda allí, rechaza o pasa al otro.



PARTE IV — CASOS DESAFÍO

Puede haber más de una respuesta válida — Justifique cada decisión

#19

DOSW Streaming — La Plataforma de Streaming

SOLID + Patrones combinados

Se construye una plataforma de streaming con: recomendaciones personalizadas por usuario, diferentes tipos de usuarios (gratis, premium, familiar), múltiples algoritmos de búsqueda (por popularidad, por relevancia, por novedad), notificaciones por múltiples canales (push, email, SMS) e integración con servicios externos (subtítulos, métricas, pagos).

PRINCIPIOS SOLID A CONSIDERAR	PATRONES A CONSIDERAR
► Single responsibility Cada componente debería tener una responsabilidad concreta. Por ejemplo, la clase de recomendaciones no debería encargarse también de enviar notificaciones. ► Open / Closes Se deben poder agregar nuevas variantes sin modificar constantemente el código existente, ya que la plataforma debería ir extendiendo funcionalidades a medida que crece. ► Dependency Inversion Nuestro núcleo de negocio debe depender de nuestras propias interfaces, no de clases concretas de terceros. Por ejemplo, si se cambia de sistema de pagos.	► Strategy Útil para recomendaciones personalizadas ya que permite inyectar la estrategia de búsqueda adecuada en tiempo de ejecución según lo que el usuario pida. ► Adapter Permite integrar servicios externos con interfaces diferentes a las que espera nuestra plataforma. ► Factory Method Puede encapsular la creación de usuarios según el tipo solicitado.



20. RETO BONUS (+0.5 puntos)

Análisis de un sistema real que uses diariamente

Elige un sistema real que uses diariamente — Netflix, Spotify, Uber, Nequi, Teams, GitHub, Instagram, ChatGPT, u otro — y realiza el siguiente análisis arquitectónico:

2 PRINCIPIOS SOLID PRESENTES Identifique dónde y por qué considera que ese principio está aplicado en el sistema real.	2 PATRONES PROBABLEMENTE USADOS Argumenta basándote en el comportamiento observable del sistema (no en su código fuente).	1 MEJORA ARQUITECTÓNICA PROPUESTA Una mejora concreta que implementarías. Justifica el impacto esperado.
--	---	--

Sistema elegido: _____ Discord _____ · La calidad de la justificación vale más que la cantidad de respuesta. Se evalúa el razonamiento, no la respuesta 'correcta'.

MI ANÁLISIS

Principios SOLID:

- **Single Responsibility:** Tienen separados el sistema de voz y el chat de texto. Si el servicio de mensajería de texto se cae, puedes seguir hablando por voz con tus amigos en los canales sin ningún problema, a menos de que se caiga el propio Discord.
- **Liskov Substitution:** Los distintos dispositivos del mismo usuario, (por ejemplo, si estoy en pc me cambio fácil al celular) se pueden intercambiar sin problema. El back de Discord espera una conexión de un cliente genérico y todos respetan el contrato de red sin romper el servicio.

Patrones:

- **Command:** Cuando utilizas los comandos (/play en un bot), la interfaz de usuario encapsula la acción y sus parámetros en un comando que el servidor recibe, procesa y finalmente ejecuta.
- **Composite:** Se evidencia en la estructura visual y de permisos del servidor. Un servidor común contiene categorías, y las categorías contienen los canales (Ej: Servidor: Warzone. Categorías: Foro, clips. Canales en cada uno de voz o texto). Todos comparten una abstracción común; gracias a esto, al silenciar o cambiar permisos en una Categoría entera, la operación se aplica a todos sus canales hijos, no toca uno por uno.

Mejora:

- Implementaría el patrón Chain of Responsibility a nivel del cliente de escritorio para mejorar la experiencia en la gestión de audio. Si estás en una partida con amigos y la torre de escritorio experimenta un fallo USB que desconecta los audífonos abruptamente, la cadena interceptaría el fallo a nivel de sistema operativo y pasaría la salida de audio automáticamente al siguiente dispositivo válido, como bocinas del pc, así evitando que la llamada o el juego se queden sin sonido de golpe, lo cual perjudica en la experiencia del juego o del entorno con amigos.



CRITERIOS DE ENTREGA

Por cada caso se evaluará la calidad del razonamiento

#	CRITERIO	PESO	ESTADO
1	Identificación correcta del principio SOLID involucrado en cada caso.	25%	☐ Pendiente
2	Justificación técnica fundamentada (no solo nombrar el principio, sino explicar por qué).	30%	☐ Pendiente
3	Propuesta de solución coherente con el principio o patrón identificado.	25%	☐ Pendiente
4	Casos desafío: análisis completo con al menos 2 principios y 2 patrones justificados.	10%	☐ Pendiente
5	Reto Bonus: sistema real con análisis argumentado (no respuesta genérica).	10% +0.5	☐ Pendiente